

03DataFrames

October 5, 2021

1 Data Science: Bridging Principles and Practice

1.1 Part 3: DataFrames

In this notebook, we will learn how to load, explore, and manipulate data in a table-like structure called a DataFrame.

1.1.1 Table of Contents

```
<li><a href='#section3'>DataFrames</a>
  <br>
  <ol type=a>
    <br>
    <li><a href='#section3a'>Reading in data</a></li>
    <br>
    <li><a href='#section3b'>Exploring data: attributes and methods </a></li>
    <br>
    <li><a href='#section3c'>Selecting columns</a></li>
    <br>
    <li><a href='#section3d'>Filtering rows</a></li>
  </ol>
</li>
</ol>
```

1.2 3. DataFrames

The last notebook covered four basic concepts of Python: expressions, names, functions, and sequences. In this next section, we'll see just how much we can do to examine and manipulate our data with only these minimal Python skills.

To work with our data, we're going to use a free Python library (collection of code) called **Pandas**. Pandas contains many many functions and other code useful for working with tabular data. The name is an abbreviation of "panel data". It is commonly used in industry and research for data analysis.

Run the next cell to import Pandas so we can use it. We're going to give Pandas the standard alias "pd" so that we don't have to type the whole name every time we want to use it.

1.3 3a. Reading in data

Real-life data is often stored in spreadsheets, including our Rocket Fuel data, which is in a *csv* (*comma-separated-values*) file.

Pandas has a function called `read_csv` which we can use to load our data. To use any Pandas function, we will use **dot notation**: first we type `pd` so the computer knows to look in the Pandas library, then we type the name of the function we want, separated by a dot.

The `read_csv` function takes one argument: a string with the location of the csv file. The location can be either a URL or a local file path.

```
In [1]: # load necessary software: THIS CELL MUST BE RUN
```

```
import pandas as pd
import numpy as np
%matplotlib inline
from gofer.ok import check
```

```
In [2]: # run this cell to load the data
```

```
ads = pd.read_csv('data/rocketfuel_data_renamed.csv', index_col=0)
```

```
# display the first 5 rows of the ads data
```

```
ads.head()
```

```
Out[2]:
```

	user id	test group	converted	total ads	most ads day	most ads hour
0	1069124	ad	False	130	Monday	20
1	1119715	ad	False	93	Tuesday	22
2	1144181	ad	False	21	Tuesday	18
3	1435133	ad	False	355	Tuesday	10
4	1015700	ad	False	276	Friday	14

Our data is now contained in a Pandas object called a **DataFrame**. The rest of this notebook deals with how to work with DataFrames.

As a reminder, here's the **data dictionary**, which explains what each column in the DataFrame contains.

user id	test group	converted	total ads	most ads day	most ads hour
The unique identifier for that user	Which testing group the user was in: "ad"- where users saw the ads (the experimental group) or "psa"- where users saw the PSAs (the control)	Whether or not the user bought a hand-bag (True if they did, False if they didn't)	The total number of ads (or PSAs) seen by that user	The day of the week on which the user saw the most ads (or PSAs)	The hour of the day during which the user saw the most ads (or PSAs)

1.4 3b. Explore the data: attributes and methods

Before we start working with the data, we should know a bit about it.

1.4.1 Attributes

Every DataFrame has **attributes** that give information about it, like the number of rows and the number of columns. Attributes are accessed using the dot method. But, since an attribute doesn't perform an operation, there are no parentheses (like there would be in a call expression that calls a function).

Here's an example. The `columns` attribute returns a list of the names of the columns in the DataFrame.

```
In [3]: # get the names of columns
ads.columns
```

```
Out[3]: Index(['user id', 'test group', 'converted', 'total ads', 'most ads day',
              'most ads hour'],
              dtype='object')
```

EXERCISE: One extremely useful attribute is `shape`, which returns the number of rows and columns in the DataFrame, separated by commas. In the next cell, get the `shape` attribute from the `ads` DataFrame.

```
In [4]: # get the shape of the DataFrame
data_shape = ads.shape
data_shape
```

```
Out[4]: (588101, 6)
```

```
In [5]: # Run this cell to check your answer for common errors
check("tests/03/shape.ok")
```

```
Out[5]: <gofer.ok.OKTestsResult at 0x7f617863f4e0>
```

1.4.2 Methods

A **method** is a lot like a function: - methods perform operations - methods are called using a set of parentheses - methods may have a set of arguments

But, a method is associated with an object like a DataFrame. Functions are not associated with any Python object.

Methods are called using dot notation. The name of the DataFrame comes before the dot, and the method comes after, followed by the parentheses. If the method takes any arguments, those go inside the parentheses.

Here's an example of the `head` method, which shows the first ten rows of the DataFrame.

```
In [6]: # show the first ten rows of the DataFrame
ads_head = ads.head()
ads_head
```

```
Out [6]:
```

	user id	test group	converted	total ads	most ads	day	hour
0	1069124	ad	False	130		Monday	20
1	1119715	ad	False	93		Tuesday	22
2	1144181	ad	False	21		Tuesday	18
3	1435133	ad	False	355		Tuesday	10
4	1015700	ad	False	276		Friday	14

EXERCISE: the `describe` method is incredibly useful for learning about your data. `describe` returns summary statistics about the numerical data in your DataFrame: things like the count of non-empty items in each column, the average, the minimum, and the maximum.

In the next cell, call the `describe` method on your DataFrame. This call will look very similar to the example where the `head` method was called; only the name of the method changes.

```
In [7]: # use dot notation to call "describe" on the ads table in place of the ellipses
ads_description = ads.describe()
ads_description
```

```
Out [7]:
```

	user id	total ads	most ads	hour
count	5.881010e+05	588101.000000	588101.000000	
mean	1.310692e+06	24.820876	14.469061	
std	2.022260e+05	43.715181	4.834634	
min	9.000000e+05	1.000000	0.000000	
25%	1.143190e+06	4.000000	11.000000	
50%	1.313725e+06	13.000000	14.000000	
75%	1.484088e+06	27.000000	18.000000	
max	1.654483e+06	2065.000000	23.000000	

```
In [8]: # Run this cell to check your answer for common errors
check("tests/03/describe.ok")
```

```
Out [8]: <gofer.ok.OKTestsResult at 0x7f6144ed2a58>
```

1.5 3c. Selecting columns

Suppose we want to answer the question of whether the ad campaign was profitable. Not all of our columns are relevant to this question, like "most ads hour". We can save computational resources and avoid confusion by *transforming* our table before we start work.

We can access a single column using **indexing**, like we did with sequences in Notebook 02. In Notebook 02, we indexed items from an array by their numerical position:

```
In [9]: # an array of prices
prices = np.array([105.99, 99.99, 119.95, 130, 124.99])

# index the item in position 2 of the array
prices[2]
```

```
Out [9]: 119.95
```

We can index a column from a DataFrame by its name. Put the string name of the column inside square brackets to the right of the DataFrame to index it.

Remember, a string must be surrounded by matching quotation marks.

```
In [10]: # index the total ads column
total_ads_col = ads["total ads"]
total_ads_col
```

```
Out[10]: 0      130
1       93
2       21
3      355
4      276
5      734
6      264
7       17
8       21
9      142
10     209
11      47
12      61
13      40
14      20
15       9
16      64
17      26
18     248
19      73
20     281
21     389
22     136
23      47
24     264
25      87
26      46
27       3
28     114
29     175
...
588071    13
588072     2
588073     1
588074     3
588075     4
588076     3
588077     4
588078     1
588079     2
588080     2
588081    16
588082     2
588083     2
```

588084	5
588085	1
588086	2
588087	2
588088	1
588089	9
588090	30
588091	2
588092	7
588093	15
588094	1
588095	2
588096	1
588097	1
588098	3
588099	1
588100	1

Name: total ads, Length: 588101, dtype: int64

EXERCISE: Use square brackets to index the "converted" column.

```
In [11]: # index the "converted" column
converted_col = ads["converted"]
converted_col
```

```
Out[11]: 0      False
         1      False
         2      False
         3      False
         4      False
         5      False
         6      False
         7      False
         8      False
         9      False
        10      False
        11      False
        12      False
        13      False
        14      False
        15       True
        16      False
        17      False
        18      False
        19      False
        20      False
        21      False
        22      False
```

```

23      False
24      False
25      False
26      False
27      False
28      False
29      False
...
588071  False
588072  False
588073  False
588074  False
588075  False
588076  False
588077  False
588078  False
588079  False
588080  False
588081  False
588082  False
588083  False
588084  False
588085  False
588086  False
588087  False
588088  False
588089  False
588090  False
588091  False
588092  False
588093  False
588094  False
588095  False
588096  False
588097  False
588098  False
588099  False
588100  False
Name: converted, Length: 588101, dtype: bool

```

```

In [12]: # Run this cell to check your answer for common errors
         check("tests/03/col-index.ok")

```

```

Out[12]: <gofer.ok.OKTestsResult at 0x7f6144ed2978>

```

1.6 3d. Filtering rows

Some analysis questions only deal with a subset of rows. How often do users convert when they saw the most ads during business hours (8AM-5PM)? What was the total number of ads seen by

the control group? Was the conversion rate greater when users saw more ads on Monday than on Tuesday?

To filter rows, we need to use **Boolean operators**. To review from Notebook 02, a **Boolean** is a data type that has only two possible values: True or False. Booleans show up in bold green in the notebook.

```
In [13]: # Boolean values
        True

        False
```

```
Out[13]: False
```

Boolean operators are operators that have the result of True or False.

Operator	Meaning	What it does
==	"equal to"	returns True if the item on the left is equal to the item on the right
>	"greater than"	returns True if the item on the left is larger than the item on the right
<	"less than"	returns True if the item on the left is smaller than the item on the right
>=	"greater than or equal to"	returns True if the item on the left is larger or the same as the item on the right
<=	"less than or equal to"	returns True if the item on the left is smaller or the same as the item on the right

Here's an example of using a Boolean operator on some numbers:

```
In [14]: # compare two numbers to see if the first one is bigger
        100 > 99
```

```
Out[14]: True
```

We can also use a Boolean operator on a column, to make a comparison for each item in the column.

```
In [15]: # get the "most ads day" column
        total_ads = ads["most ads day"]

        # show the first 5 items of the "most ads day" column
        total_ads.head()
```

```
Out[15]: 0    Monday
        1    Tuesday
        2    Tuesday
        3    Tuesday
        4    Friday
        Name: most ads day, dtype: object
```



```
In [16]: # check whether the day someone saw the most ads was Monday
total_ads == "Monday"
```

```
Out[16]: 0          True
         1          False
         2          False
         3          False
         4          False
         5          False
         6          False
         7          False
         8          False
         9          True
        10          False
        11          False
        12          False
        13          False
        14          False
        15          False
        16          False
        17          False
        18          False
        19          False
        20          False
        21          False
        22          False
        23          False
        24          False
        25          False
        26          False
        27          False
        28          True
        29          False
        ...
    588071          False
    588072          False
    588073          False
    588074          False
    588075          False
    588076          False
    588077          False
    588078          False
    588079          False
    588080          False
    588081          False
    588082          False
    588083          False
    588084          False
```

```

588085    False
588086    False
588087    False
588088    False
588089    False
588090    False
588091    False
588092    False
588093    False
588094    False
588095    False
588096    False
588097    False
588098    False
588099    False
588100    False
Name: most ads day, Length: 588101, dtype: bool

```

If we index a DataFrame or column by an array of Booleans, we will get only the rows or items for which the condition is True.

```

In [17]: # get rows with users who saw the most ads on Monday
monday_ads = ads[ads["most ads day"] == "Monday"]

```

This syntax can be a bit confusing. Note that * the part outside of the square brackets to the left is the DataFrame out of which we want to filter rows * everything in the square brackets is the condition we want to use to filter rows

Here's another example. Outside of the brackets is the ads DataFrame, from which we want only certain rows. Inside the brackets is the condition to select the rows: we want to see only the users who saw more than 500 ads, so we're selecting the "total ads" column and using > to see which items have a value of more than 500.

```

In [18]: # example 2: get all rows that had more than 500 total ads
ads[ads["total ads"] > 500]

```

```

Out[18]:
   user id test group  converted  total ads most ads day  most ads hour
5      1137664      ad    False      734    Saturday         10
48     1381174      ad    False      588    Saturday         20
64     1261130      ad    False     1057   Thursday         20
72     1474691      ad    False      738  Wednesday          4
94     1447938      ad    False      593     Friday         15
107    1389878      ad     True     1328    Saturday         19
137    1434278      ad    False      653    Saturday         12
154    1218677      ad    False      555     Monday          8
175    1108047      ad    False      505   Thursday          8
176    1356436      ad    False      640     Tuesday         12
185    1204725      ad    False      983     Sunday         19
200    1058604      ad    False      720  Wednesday         22
212    1555915      ad    False     1056     Friday         12

```

266	1144069	ad	False	681	Wednesday	15
269	1073813	ad	False	733	Wednesday	19
316	1282386	ad	False	516	Monday	19
362	1334292	ad	False	633	Wednesday	10
402	1455029	ad	True	657	Sunday	11
428	1381222	ad	False	526	Tuesday	4
429	1263606	ad	False	607	Friday	8
506	1086307	ad	False	1632	Friday	20
529	1439612	ad	False	525	Saturday	10
589	1013777	ad	False	587	Friday	12
633	1615405	ad	False	535	Saturday	8
662	1513479	ad	False	568	Wednesday	20
673	1174355	ad	True	720	Monday	13
699	1222430	ad	False	617	Monday	18
714	1278985	ad	False	655	Friday	1
831	1321038	ad	False	700	Monday	17
844	1445554	ad	False	1015	Friday	11
...	...	...	...	...	...	...
130109	1454698	ad	False	535	Thursday	13
130183	1211439	ad	False	628	Tuesday	13
130241	1562297	ad	False	518	Monday	15
131298	1581498	ad	False	921	Thursday	15
131922	1487013	ad	False	570	Thursday	10
132273	1393392	ad	False	541	Thursday	14
133029	1329455	ad	True	689	Tuesday	12
133405	1073464	ad	False	579	Wednesday	1
137214	1449282	ad	False	569	Tuesday	21
138763	915795	psa	False	505	Thursday	14
139719	1558152	ad	False	673	Thursday	11
139929	1345733	ad	True	791	Thursday	10
140059	1608955	ad	False	733	Monday	14
141321	1651866	ad	True	952	Thursday	12
141796	1448448	ad	False	596	Tuesday	10
141889	1337947	ad	False	808	Wednesday	13
142674	1204856	ad	True	625	Thursday	14
142954	1373958	ad	False	518	Friday	14
143013	1372425	ad	True	772	Wednesday	14
144248	1587752	ad	True	603	Tuesday	8
144360	1487252	ad	True	728	Thursday	9
148712	1307663	ad	False	729	Wednesday	10
149379	1615292	ad	True	554	Wednesday	14
150538	1547978	ad	False	744	Wednesday	15
151321	1617878	ad	True	522	Wednesday	14
155124	1368486	ad	False	502	Thursday	14
155489	1317131	ad	False	711	Wednesday	20
157902	1376095	ad	True	512	Friday	19
165490	1596488	ad	False	633	Friday	14
180840	1462116	ad	True	663	Monday	12

[585 rows x 6 columns]

EXERCISE: Oftentimes, we want to calculate statistics separately for the control and experimental groups. Create two tables, one containing only rows where the user was in the "ad" group (the experimental group) and one with only rows where the user was in the "psa" group (the control group).

Hint: We've given you the code to create this for the experiment group. Fill in the appropriate value to select users who did NOT see ads.

```
In [19]: # users in the experiment group
experiment = ads[ads["test group"] == "ad"]
experiment.head()
```

```
Out[19]:
```

	user id	test group	converted	total ads	most ads day	most ads hour
0	1069124	ad	False	130	Monday	20
1	1119715	ad	False	93	Tuesday	22
2	1144181	ad	False	21	Tuesday	18
3	1435133	ad	False	355	Tuesday	10
4	1015700	ad	False	276	Friday	14

```
In [20]: # users in the control group
# fill in the ellipses with the correct text to select users who saw psas
control = ads[ads["test group"] == "psa"]
control.head()
```

```
Out[20]:
```

	user id	test group	converted	total ads	most ads day	most ads hour
18	900681	psa	False	248	Saturday	19
38	905704	psa	False	27	Thursday	8
68	904595	psa	False	13	Tuesday	19
140	901904	psa	False	32	Wednesday	19
157	902234	psa	False	105	Tuesday	19

```
In [21]: # Run this cell to check your answer for common errors
check("tests/03/control-group.ok")
```

```
Out[21]: <gofer.ok.OKTestsResult at 0x7f61782aa2e8>
```

References

- Rocket Fuel data and discussion questions adapted from materials by Zsolt Katona and Brian Bell, BerkeleyHaas Case Series

Author: Keeley Takimoto